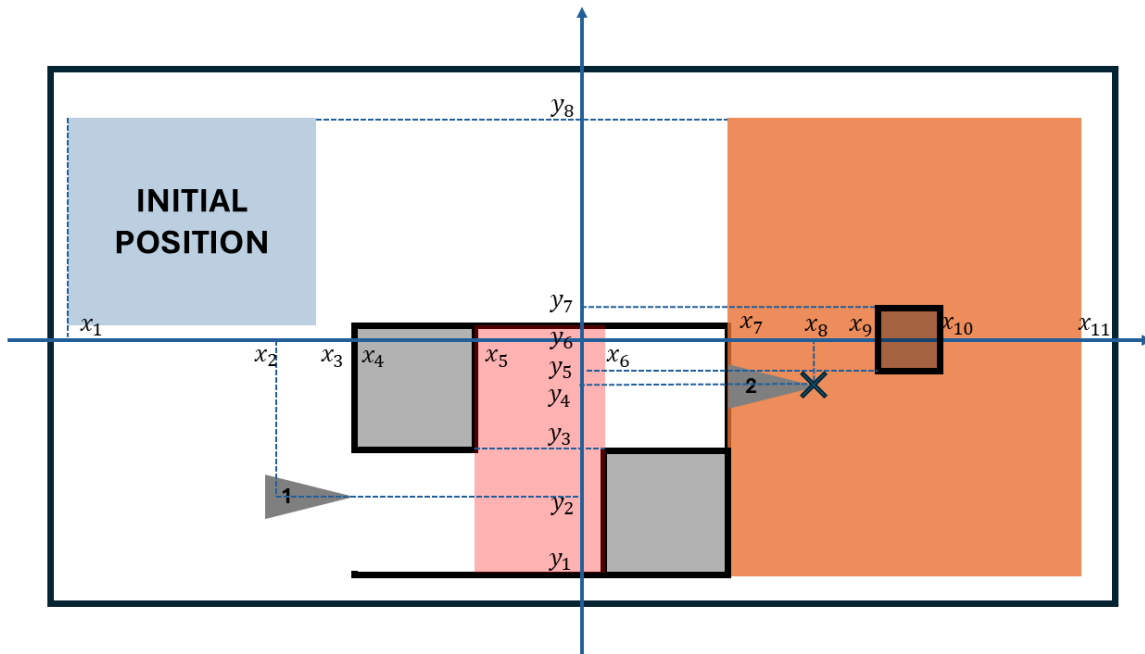


Final Activity

The final activity is composed of 4 tasks of increasing difficulty. Activities must be carried out first in simulation, and then on the actual TurtleBot. Figure 1 reports a map of the environment. Numerical values of the coordinates are reported in the table below.



VARIABLE NAME	VALUE
x_1	-2.90 [m]
x_2	-1.60 [m]
x_3	-1.50 [m]
x_4	-1.40 [m]
x_5	-0.60 [m]
x_6	0.20 [m]
x_7	1.00 [m]
x_8	1.45 [m]
x_9	1.80 [m]
x_{10}	2.20 [m]
x_{11}	2.90 [m]
y_1	-1.50 [m]
y_2	-1.10 [m]
y_3	-0.70 [m]
y_4	-0.30 [m]
y_5	-0.20 [m]
y_6	0.10 [m]
y_7	0.20 [m]
y_8	1.50 [m]

You can use measures from wheels encoders, IMU, and the motion capture (position and orientation). Regarding motion capture measures:

- In the red area ($x_5 \leq x \leq x_6$ and $y_1 \leq y \leq y_5$), measurements are corrupted by Gaussian noise with standard deviation $\sigma_e = 0.02$;
- In the orange area ($x_7 \leq x \leq x_9$ and $y_1 \leq y \leq y_6$) $p_{loss} = 0.9$, namely, on average, every 10 measures 9 are corrupted.
- In the remaining area you can assume that there is no noise on the motion capture measures. However, in the actual system there could be random errors due to occlusions and light effects.

To interact with the motion capture, we will provide you a Simulink block that implements the behavior described. You will have to set the connection parameters. The block returns:

- **Y_vicon_turtulbot**, a 3 dimensional vector with the raw robot configuration $[x, y, \theta]$ measured from the Vicon (without calibration) and available to the robot. If a measure is not available because you are in the red or orange area, you will read $[0,0,0]$. So, **to check for missing measurements, you can just check if Y_vicon_turtulbot is equal to $[0, 0, 0]$.**
 - **Y_vicon_ground_truth**, a 3 dimensional vector with the raw robot configuration $[x, y, \theta]$ measured from the Vicon (without calibration). **This variable can not be used by the robot. You can use this variable only as a ground truth** to analyze and tune your algorithms and report results.

To perform the task, you can use all the code developed during numerical and experimental activities, including identification and calibration procedures. You can design a single Simulink file that implements TASK 1, TASK 2, and TASK 3. Or write distinct Simulink files for each task. **Simulate the all the tasks before testing them on the actual system.** In the final report you will have to include:

- comparisons between simulations and actual experiments ;
- parameter tuning performed in simulation and on the actual system;
- performance of the controllers (actual vs desired state and tracking/regulation error);
- performance of the localization (actual vs estimated state).

Do not ask for solutions, but you can discuss with the teacher and the assistant your strategy and any potential issues.

As regards saturation and sampling time, consider the following values:

- $\omega_M = 10 \text{ [rad/sec]}$;
- $T_s = 0.04 \text{ [sec]}$.

In the following, we describe the tasks to perform.

TASK 1

Place the robot in the area “INITIAL POSITION” ($x_1 \leq x \leq x_3$ and $y_6 \leq y \leq y_8$), with orientation close to $-\frac{\pi}{2}$. Move the robot to *Configuration 1*:

$$q = \begin{bmatrix} x_2 \\ y_2 \\ 0 \end{bmatrix}.$$

Avoid collisions with the environment. Perform the task starting from 4 different initial conditions in “INITIAL POSITION”

TASK 2

Move the robot to the point with cartesian coordinates $x = x_7$ and $y = y_4[m]$, starting from the configuration reached in TASK 1. Design a trajectory that avoids collisions with walls and obstacles. Develop a localization strategy that handle the fact that in the red area motion capture measures are noisy. Use a suitable control strategy.

TASK 3

Designs a circular trajectory that makes one and a half turns around the cube positioned at the center of the orange area, starting from the cartesian coordinates $x = x_8$ and $y = y_4$.

TASK 4 (optional, can be solved also just in simulation)

Follow the same trajectory of TASK 3, starting from a configuration close to the final configuration of TASK 2 (you can manually place the robot). Now motion capture is not available. You can use wheels encoders, IMU, and lidar for localization. The lidar returns distances between its center and the walls of the cube positioned in the center of the orange area, computed along known directions. The coordinates of the cube are known. More specifically:

- The lidar is translated w.r.t. the kinematic center of the Turtlebot along the sagittal axis of $0.076 [m]$.
- The lidar returns a vector of distances $d_0 \dots d_{n-1}$ and the correspondent angles $\theta_0 \dots \theta_{n-1}$. Figure 2 reports an example with $n = 8$;

Simulate the system and elaborate a strategy to exploit lidar measures for localization. Analyze effectiveness of the proposed strategy, also considering errors on the initial

configuration, namely, consider the case that the initial configuration of you localization algorithm is different from the actual one. If you derive an effective strategy test the control with localization.

Hint: write the nonlinear measurement model of the lidar, and use EKF for localization.

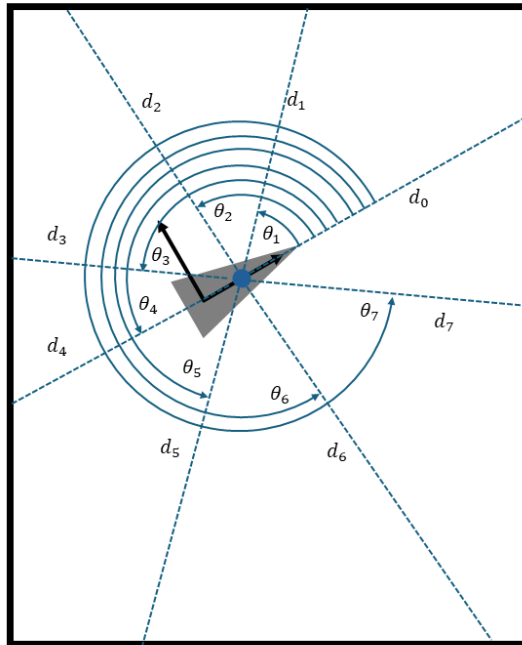


Figure 2: schematization of the lidar measures.